



VIT[®]
BHOPAL

FUNDAMENTALS OF AI AND ML



A PROJECT REPORT ON **'LIBRARY MANAGEMENT SYSTEM'**

Submitted By – Naina Jha

Registration No. – 25BCE11087

Slot – A21+A22+A23

INTRODUCTION

In today's fast-paced academic environment, students' performance is influenced by many different factors, not just how much they study. Elements like attendance, sleep patterns, consistency in completing assignments, and even screen time play an important role in shaping a student's overall results. Understanding how these factors affect performance can help students improve their study habits and skills, and achieve better outcomes.

The **Student Performance Predictor** is designed as a simple and interactive web-based application that aims to analyze and predict a student's marks based on a combination of academic and lifestyle inputs. Instead of relying on a single factor, this project takes a more practical approach by considering multiple real-life aspects such as study hours, previous exam performance, attendance percentage, sleep duration, and screen time. This makes the system more realistic and closer to how performance is evaluated in real-world scenarios.

The main idea behind this project is to introduce basic concepts of Machine Learning, such as prediction and data analysis, in a simple and understandable way. Although the system does not use a fully trained machine learning model, it simulates the behavior of one by using a weighted formula

to generate predictions. This helps in building a strong foundation for understanding how real ML models work.

In addition to predicting marks, the system also focuses on providing meaningful insights. It includes features like graphical representation of performance, trend analysis (showing whether performance is improving or declining), and AI-based suggestions. These suggestions guide the user on how they can improve, for example by increasing study hours, improving attendance, getting enough sleep, or reducing distractions.

One of the most important features of this project is the **Detailed Analysis**, which gives a structured report including performance level, strengths, weaknesses, and personalized recommendations. This makes the system not just a predictor, but also a helpful tool for self-evaluation and improvement.

Overall, this project combines prediction, visualization, and intelligent feedback into a single platform. It demonstrates how technology and basic AI concepts can be used to solve real-life problems in education. The project is simple to use, yet powerful enough to provide valuable insights, making it both educational and practical.

Objective

The main objective of this project is to create a simple yet effective system that can predict a student's academic performance based on a combination of academic and lifestyle factors. Instead of focusing only on marks or study time, this project aims to show that a student's performance is influenced by multiple real-life habits such as attendance, sleep, consistency in completing assignments, and even screen time.

Another important objective of this project is to help students become more aware of their own study patterns and daily routines. By entering their data into the system and observing the predicted results, students can better understand how their habits are affecting their performance and marks. This awareness can motivate them to make better decisions, such as increasing study time, improving attendance, completing assignments or reducing distractions.

The project also aims to introduce the basic concepts of Machine Learning in a simple and practical way. Although it does not use a fully trained model, it demonstrates how prediction works using a logical formula based on weighted factors. This helps in building a strong foundation for

understanding more advanced AI and Machine Learning techniques in the future.

In addition to prediction, the system is designed to provide meaningful feedback. One of the key objectives is to offer AI-based suggestions that guide the user toward improvement. These suggestions are generated based on the input data and performance trends, making the system more interactive and useful rather than just informative.

Another objective is to present the results in a clear and visually appealing manner. The use of graphs allows users to track their performance over multiple attempts and understand trends easily. This visual representation makes the system more engaging and helps users quickly identify whether their performance is improving or declining over time.

The project also focuses on creating a clean and user-friendly interface so that anyone can use it without difficulty. The goal is to make the system simple, interactive, and accessible, even for users who may not have a technical background.

Finally, this project aims to combine prediction, visualization, and intelligent feedback into one platform. By doing so, it demonstrates how basic AI concepts can be applied to solve real-world problems in education and how technology can support students in improving their academic performance.

Requirement analysis

Student Performance Predictor is designed to be a simple and user-friendly system that helps in analyzing and predicting student performance. To build this system effectively, it is important to understand both the functional and non-functional requirements given below:

1. Functional Requirements:- The system should allow the user to enter different inputs that affect academic performance. These inputs include study hours, attendance percentage, assignment completion, previous marks, sleep hours, and screen time. After entering these values, the user should be able to click a button to predict the marks.

The system must calculate the predicted marks using a predefined formula and display the result clearly. It should also provide instant feedback in the form of AI-based suggestions to guide the user on how to improve their performance.

Another important requirement is the graphical representation of data. The system should display a line graph that shows the predicted marks over multiple attempts. This helps users track their progress and understand performance trends.

Additionally, the system should include a feature for detailed analysis. When the user clicks on the “Get Detailed Analysis”

button, it should generate a structured report that includes performance level, strengths, weaknesses, and recommendations.

The system should also provide a reset option to clear all inputs and graph data, allowing the user to start fresh.

2. Non-Functional Requirements:-The system should have a clean and simple user interface so that it is easy to use for anyone, even without technical knowledge. The design should be visually appealing and organized in a way that improves user experience.

It should be fast and responsive, meaning the results and graph updates should appear instantly after user input. The system must also be reliable, giving consistent outputs for the same inputs.

Compatibility is another important factor. The application should run smoothly on any modern web browser without requiring any special software installation.

Lastly, the system should be easy to maintain and extend. Future improvements, such as adding a real machine learning model or more features, should be possible without major changes.

Overall, the requirement of this project is to create a system that is not only capable of predicting marks but also helps users understand and improve their performance through analysis, visualization, and smart suggestions.

Top-Down Design

The **Student Performance Predictor** is designed using a top-down approach, which means the system is first divided into major components and then each component is further broken down into smaller and manageable parts. This approach helps in better understanding, organizing, and developing the project step by step and also helps to improve skills.

1. Main System (Student Performance Predictor):-

At the highest level, the system works as a complete application that takes user inputs, processes them, and provides results along with analysis and visualization.

2. Input Module:-

This is the first part of the system where the user interacts with the application.

The user is required to enter:

- Study Hours
- Attendance (%)
- Assignment Completion (%)
- Previous Marks (%)
- Sleep Hours
- Screen Time

This module ensures that all inputs are collected properly and are ready for processing.

3. Processing Module:-

After receiving the inputs, the system processes the data using a predefined formula.

This module is responsible for:

- Calculating the predicted marks
- Ensuring the result stays within a valid range (0–100)
- Storing the result for further use

This is the core part of the system where the prediction actually happens.

4. Output Module:-

Once the prediction is complete, the system displays the results to the user.

This includes:

- Predicted Marks
- Performance Trend (Improving or Declining)

The output is shown clearly so that the user can easily understand their performance.

5. Visualization Module:-

This module is responsible for displaying the graph.

It:

- Plots predicted marks for each attempt
- Updates the graph dynamically
- Helps the user analyze performance trends visually

This makes the system more interactive and easy to understand.

6. AI Suggestion Module:-

This part provides quick suggestions based on the user's input and predicted performance.

For example:

- Increase study hours
- Improve attendance
- Reduce screen time

These suggestions help users identify areas of improvement instantly.

7. Detailed Analysis Module:-

This module provides a deeper understanding of the performance.

When the user clicks on the “**Get Detailed Analysis**” button, the system generates:

- Performance Level
- Strengths
- Weaknesses
- Recommendations

This makes the system more informative and useful for self-evaluation.

8. Control Module:-

This module manages user interactions such as:

- Predict button
- Detailed analysis button
- Reset button

It ensures smooth functioning and proper flow of the system.

In this project, the top-down design helps in dividing the system into clear modules like input, processing, output, visualization, and analysis. This structured approach makes the system easy to develop, understand, and maintain while ensuring that all functionalities work together smoothly.

Main Code

➤ index.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width,  
initial-scale=1.0">
```

```
  <title>Student Performance Dashboard</title>
```

```
  <link rel="stylesheet" href="style.css">
```

```
  <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
```

```
</head>
```

```
<body>
```

```
<div class="container">
```

```
  <h1> Student Performance Dashboard</h1>
```

```
  <div class="form-container">
```

```
    <label>Study Hours</label>
```

```
    <input type="number" id="studyHours">
```

```
<label>Attendance (%)</label>
<input type="number" id="attendance">
<label>Assignments Completion (%)</label>
<input type="number" id="assignments">
<label>Previous Marks (%)</label>
<input type="number" id="previousMarks">
<label>Sleep Hours</label>
<input type="number" id="sleepHours">
<label>Screen Time (hrs/day)</label>
<input type="number" id="screenTime">
<button id="predictBtn">Predict Marks</button>
</div>
<div class="result-container">
  <h2>Predicted Marks: <span id="predictedMarks">--
</span></h2>
  <p id="trendText"></p>
  <div id="aiSuggestion" class="ai-box"></div>
  <button id="detailBtn" class="detail-btn">Get Detailed
Analysis</button>
  <div id="detailedSuggestion" class="detail-box"></div>
```

</div>

<canvas id="marksChart"></canvas>

<div class="center-btn">

 <button id="resetBtn" class="reset-btn">Reset
Graph</button>

</div>

</div>

<script src="script.js"></script>

</body>

</html>

```
File Edit Selection View Go Run Terminal Help Python Code
EXPLORER
  index.html
  README.md
  script.js
  style.css
  OUTLINE
  TIMELINE
index.html X
  index.html > html > body > div.container > h1
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-scale=1.0">
6   <title>Student Performance Dashboard</title>
7   <link rel="stylesheet" href="style.css">
8   <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
9 </head>
10 <body>
11
12 <div class="container">
13   <h1> Student Performance Dashboard</h1>
14
15   <div class="form-container">
16     <label>Study Hours</label>
17     <input type="number" id="studyHours">
18
19     <label>Attendance (%)</label>
20     <input type="number" id="attendance">
21
22     <label>Assignments Completion (%)</label>
23     <input type="number" id="assignments">
24
25     <label>Previous Marks (%)</label>
26     <input type="number" id="previousMarks">
27
28     <label>Sleep Hours</label>
29     <input type="number" id="sleepHours">
30
31     <label>Screen Time (hrs/day)</label>
32     <input type="number" id="screenTime">
33
34     <button id="predictBtn">Predict Marks</button>
35   </div>
36
37   <div class="result-container">
```

```
File Edit Selection View Go Run Terminal Help Python Code
EXPLORER
  index.html
  README.md
  script.js
  style.css
  OUTLINE
  TIMELINE
index.html X
  index.html > html
2 <html lang="en">
10 <body>
12 <div class="container">
15 <div class="form-container">
20 <input type="number" id="previousMarks">
27
28 <label>Sleep Hours</label>
29 <input type="number" id="sleepHours">
30
31 <label>Screen Time (hrs/day)</label>
32 <input type="number" id="screenTime">
33
34 <button id="predictBtn">Predict Marks</button>
35 </div>
36
37 <div class="result-container">
38 <h2>Predicted Marks: <span id="predictedMarks">--</span></h2>
39 <p id="trendText"></p>
40
41 <div id="aiSuggestion" class="ai-box"></div>
42
43 <button id="detailBtn" class="detail-btn">Get Detailed Analysis</button>
44 <div id="detailedSuggestion" class="detail-box"></div>
45 </div>
46
47 <canvas id="marksChart"></canvas>
48
49 <div class="center-btn">
50   <button id="resetBtn" class="reset-btn">Reset Graph</button>
51 </div>
52 </div>
53
54 <script src="script.js"></script>
55 </body>
56 </html>
```

➤ script.js

```
const predictBtn = document.getElementById('predictBtn');
const resetBtn = document.getElementById('resetBtn');
const detailBtn = document.getElementById('detailBtn');
const predictedMarksSpan =
document.getElementById('predictedMarks');
const trendText = document.getElementById('trendText');
const aiSuggestion = document.getElementById('aiSuggestion');
const detailedSuggestion =
document.getElementById('detailedSuggestion');
let marksData = [];
let lastInputs = {};
// Prediction
function predictMarks(data) {
  let marks =
    (0.25 * data.previousMarks) +
    (5 * data.studyHours) +
    (0.25 * data.attendance) +
    (0.15 * data.assignments) +
```



```
(2 * data.sleepHours) -  
(2 * data.screenTime);  
return Math.max(0, Math.min(100, marks));  
}  
  
// Quick Suggestion  
function generateSuggestion(data, marks) {  
    if (marks < 40) return "Poor performance. Focus more.";  
    if (marks < 70) return "Average performance. Improve  
consistency.";  
    return "Excellent performance. Keep it up!";  
}  
  
// Detailed AI Report  
function generateDetailedSuggestion(data, marks) {  
    let performance =  
        marks < 40 ? "Poor" :  
        marks < 70 ? "Average" : "Excellent";  
    let strengths = [];  
    let weaknesses = [];  
    if (data.studyHours >= 4) strengths.push("Good study hours");
```

```
    else weaknesses.push("Low study hours");  
    if (data.attendance >= 75) strengths.push("Good  
attendance");  
    else weaknesses.push("Low attendance");  
    if (data.sleepHours >= 6) strengths.push("Healthy sleep");  
    else weaknesses.push("Insufficient sleep");  
    if (data.screenTime <= 5) strengths.push("Controlled screen  
time");  
    else weaknesses.push("High screen time");  
    return `
```

```
<div class="section">
```

```
<span class="section-title"> Performance Level:</span>  
${performance}
```

```
</div>
```

```
<div class="section">
```

```
<span class="section-title"> Predicted Marks:</span>  
${marks.toFixed(2)}
```

```
</div>
```

```
<div class="section">
```

```
<span class="section-title"> Strengths:</span>
```

```
- ${strengths.join("\n- ") || "None"}
```

```
</div>
```

```
<div class="section">
```

```
<span class="section-title">⚠ Weaknesses:</span>
```

```
- ${weaknesses.join("\n- ") || "None"}
```

```
</div>
```

```
<div class="section">
```

```
<span class="section-title"> Recommendations:</span>
```

```
- Improve weak areas
```

```
- Maintain consistency
```

```
- Balance study and rest
```

```
</div>
```

```
`;
```

```
}
```

```
// Chart
```

```
const ctx =
```

```
document.getElementById('marksChart').getContext('2d');
```

```
let chart = new Chart(ctx, {
```

```
  type: 'line',
```

```
data: {
  labels: [],
  datasets: [{
    label: 'Marks',
    data: [],
    borderColor: '#2563eb',
    tension: 0.4
  }]
},
options: {
  scales: {
    y: { min: 0, max: 100 }
  }
}
});

// Predict
predictBtn.addEventListener('click', () => {
  const data = {
```

```
        studyHours:
+document.getElementById('studyHours').value,
        attendance:
+document.getElementById('attendance').value,
        assignments:
+document.getElementById('assignments').value,
        previousMarks:
+document.getElementById('previousMarks').value,
        sleepHours:
+document.getElementById('sleepHours').value,
        screenTime:
+document.getElementById('screenTime').value
    };
    if (Object.values(data).some(isNaN)) {
        alert("Fill all fields!");
        return;
    }
    const result = predictMarks(data);
    predictedMarksSpan.textContent = result.toFixed(2);
    marksData.push(result);
    lastInputs = { data, result };
```

```
chart.data.labels.push(`Attempt ${marksData.length}`);
chart.data.datasets[0].data.push(result);
chart.update();
aiSuggestion.textContent = generateSuggestion(data, result);
if (marksData.length > 1) {
  trendText.textContent =
    result > marksData[marksData.length - 2]
      ? "Trend: Improving "
      : "Trend: Declining ";
}
});

// Detailed Button
detailBtn.addEventListener('click', () => {
  if (!lastInputs.result) return alert("Predict first!");
  detailedSuggestion.innerHTML =
    generateDetailedSuggestion(lastInputs.data,
lastInputs.result);
});

// Reset
```

```

resetBtn.addEventListener('click', () => {

    marksData = [];

    chart.data.labels = [];

    chart.data.datasets[0].data = [];

    chart.update();

    predictedMarksSpan.textContent = "--";

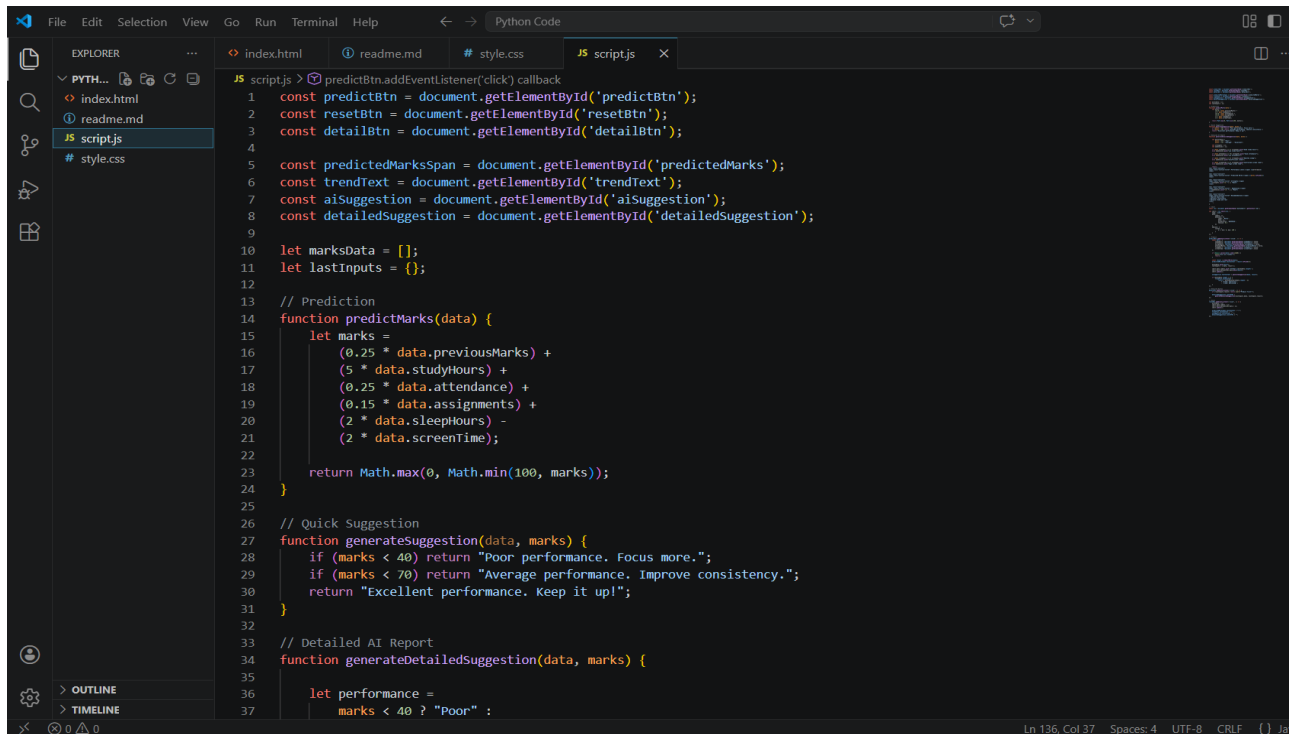
    trendText.textContent = "";

    aiSuggestion.textContent = "";

    detailedSuggestion.innerHTML = "";

});

```



```

VS Code Explorer:
- index.html
- README.md
- script.js
- style.css

script.js:
1  const predictBtn = document.getElementById('predictBtn');
2  const resetBtn = document.getElementById('resetBtn');
3  const detailBtn = document.getElementById('detailBtn');
4
5  const predictedMarksSpan = document.getElementById('predictedMarks');
6  const trendText = document.getElementById('trendText');
7  const aiSuggestion = document.getElementById('aiSuggestion');
8  const detailedSuggestion = document.getElementById('detailedSuggestion');
9
10 let marksData = [];
11 let lastInputs = {};
12
13 // Prediction
14 function predictMarks(data) {
15     let marks =
16         (0.25 * data.previousMarks) +
17         (5 * data.studyHours) +
18         (0.25 * data.attendance) +
19         (0.15 * data.assignments) +
20         (2 * data.sleepHours) -
21         (2 * data.screenTime);
22
23     return Math.max(0, Math.min(100, marks));
24 }
25
26 // Quick Suggestion
27 function generateSuggestion(data, marks) {
28     if (marks < 40) return "Poor performance. Focus more.";
29     if (marks < 70) return "Average performance. Improve consistency.";
30     return "Excellent performance. Keep it up!";
31 }
32
33 // Detailed AI Report
34 function generateDetailedSuggestion(data, marks) {
35
36     let performance =
37         marks < 40 ? "Poor" :

```

```
File Edit Selection View Go Run Terminal Help Python Code
EXPLORER
  index.html
  README.md
  JS script.js
  style.css
  OUTLINE
  TIMELINE
  0 0 0

JS script.js > predictBtn.addEventListener('click') callback
34 function generateDetailedSuggestion(data, marks) {
35     let performance =
36         marks < 40 ? "Poor" :
37         marks < 70 ? "Average" : "Excellent";
38
39     let strengths = [];
40     let weaknesses = [];
41
42     if (data.studyHours >= 4) strengths.push("Good study hours");
43     else weaknesses.push("Low study hours");
44
45     if (data.attendance >= 75) strengths.push("Good attendance");
46     else weaknesses.push("Low attendance");
47
48     if (data.sleepHours >= 6) strengths.push("Healthy sleep");
49     else weaknesses.push("Insufficient sleep");
50
51     if (data.screenTime <= 5) strengths.push("Controlled screen time");
52     else weaknesses.push("High screen time");
53
54     return `
55     <div class="section">
56     <span class="section-title"> Performance Level:</span> ${performance}
57     </div>
58     <div class="section">
59     <span class="section-title"> Predicted Marks:</span> ${marks.toFixed(2)}
60     </div>
61     <div class="section">
62     <span class="section-title"> Strengths:</span>
63     - ${strengths.join("\n- ") || "None"}
64     </div>
65     <div class="section">
66     <span class="section-title"> Weaknesses:</span>
67     - ${weaknesses.join("\n- ") || "None"}
68     `;
69 }
70
71
```

```
File Edit Selection View Go Run Terminal Help Python Code
EXPLORER
  index.html
  README.md
  JS script.js
  style.css
  OUTLINE
  TIMELINE
  0 0 0

JS script.js > predictBtn.addEventListener('click') callback
71 - ${weaknesses.join("\n- ") || "None"}
72 </div>
73
74 <div class="section">
75 <span class="section-title"> Recommendations:</span>
76 - Improve weak areas
77 - Maintain consistency
78 - Balance study and rest
79 </div>
80 `;
81 }
82
83 // Chart
84 const ctx = document.getElementById('marksChart').getContext('2d');
85
86 let chart = new Chart(ctx, {
87     type: 'line',
88     data: {
89         labels: [],
90         datasets: [{
91             label: 'Marks',
92             data: [],
93             borderColor: '#2563eb',
94             tension: 0.4
95         }]
96     },
97     options: {
98         scales: {
99             y: { min: 0, max: 100 }
100         }
101     }
102 });
103
104 // Predict
105 predictBtn.addEventListener('click', () => {
106     const data = {
107
108     }
109 }
110
```



```
File Edit Selection View Go Run Terminal Help Python Code
EXPLORER
PYTHON...
index.html
readme.md
JS script.js
style.css

JS script.js > predictBtn.addEventListener('click') callback
105 predictBtn.addEventListener('click', () => {
106     const data = {
107         studyHours: +document.getElementById('studyHours').value,
108         attendance: +document.getElementById('attendance').value,
109         assignments: +document.getElementById('assignments').value,
110         previousMarks: +document.getElementById('previousMarks').value,
111         sleepHours: +document.getElementById('sleepHours').value,
112         screenTime: +document.getElementById('screenTime').value
113     };
114
115     if (Object.values(data).some(isNaN)) {
116         alert("Fill all fields!");
117         return;
118     }
119
120     const result = predictMarks(data);
121     predictedMarksSpan.textContent = result.toFixed(2);
122
123     marksData.push(result);
124     lastInputs = { data, result };
125
126     chart.data.labels.push(`Attempt ${marksData.length}`);
127     chart.data.datasets[0].data.push(result);
128     chart.update();
129
130     aiSuggestion.textContent = generateSuggestion(data, result);
131
132     if (marksData.length > 1) {
133         trendText.textContent =
134             result > marksData[marksData.length - 2]
135                 ? "Trend: Improving "
136                 : "Trend: Declining ";
137     }
138 });
139
140 // Detailed Button
141 detailBtn.addEventListener('click', () => {
```

```
130     aiSuggestion.textContent = generateSuggestion(data, result);
131
132     if (marksData.length > 1) {
133         trendText.textContent =
134             result > marksData[marksData.length - 2]
135                 ? "Trend: Improving "
136                 : "Trend: Declining ";
137     }
138 });
139
140 // Detailed Button
141 detailBtn.addEventListener('click', () => {
142     if (!lastInputs.result) return alert("Predict first!");
143
144     detailedSuggestion.innerHTML =
145         generateDetailedSuggestion(lastInputs.data, lastInputs.result);
146 });
147
148 // Reset
149 resetBtn.addEventListener('click', () => {
150     marksData = [];
151     chart.data.labels = [];
152     chart.data.datasets[0].data = [];
153     chart.update();
154
155     predictedMarksSpan.textContent = "--";
156     trendText.textContent = "";
157     aiSuggestion.textContent = "";
158     detailedSuggestion.innerHTML = "";
159 });
```



style.css

```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}  
  
body {  
  font-family: 'Segoe UI', sans-serif;  
  background: linear-gradient(135deg, #eef2f7, #d9e4f5);  
  min-height: 100vh;  
  display: flex;  
  justify-content: center;  
  align-items: center;  
}  
  
.container {  
  width: 520px;  
  background: #ffffff;  
  padding: 35px 30px;  
  border-radius: 20px;
```

```
    box-shadow: 0 25px 60px rgba(0, 0, 0, 0.1);
}

h1 {
    text-align: center;
    margin-bottom: 25px;
    font-size: 26px;
}

.form-container {
    display: flex;
    flex-direction: column;
    gap: 10px;
}

label {
    font-size: 13px;
}

input {
    padding: 11px;
    border-radius: 10px;
    border: 1px solid #ccc;
```

```
}
```

```
button {
```

```
    padding: 12px;
```

```
    border-radius: 10px;
```

```
    border: none;
```

```
    cursor: pointer;
```

```
}
```

```
/* Buttons */
```

```
#predictBtn {
```

```
    background: #2563eb;
```

```
    color: white;
```

```
}
```

```
.detail-btn {
```

```
    background: #16a34a;
```

```
    color: white;
```

```
    margin-top: 10px;
```

```
}
```

```
.reset-btn {
```

```
    background: #dc2626;
```

```
    color: white;
}

.center-btn {
    display: flex;
    justify-content: center;
    margin-top: 15px;
}

/* Result */
.result-container {
    margin-top: 20px;
    padding: 15px;
    background: #f8fafc;
    border-radius: 12px;
    text-align: center;
}

/* AI Box */
.ai-box {
    padding: 10px;
    background: #eff6ff;
```

```
border-left: 4px solid #2563eb;
border-radius: 8px;
margin-top: 8px;
}
```

```
/* Detailed Report */
```

```
.detail-box {
    margin-top: 10px;
    padding: 15px;
    background: #f0fdf4;
    border-left: 5px solid #16a34a;
    border-radius: 10px;
    font-size: 13px;
    text-align: left;
}
```

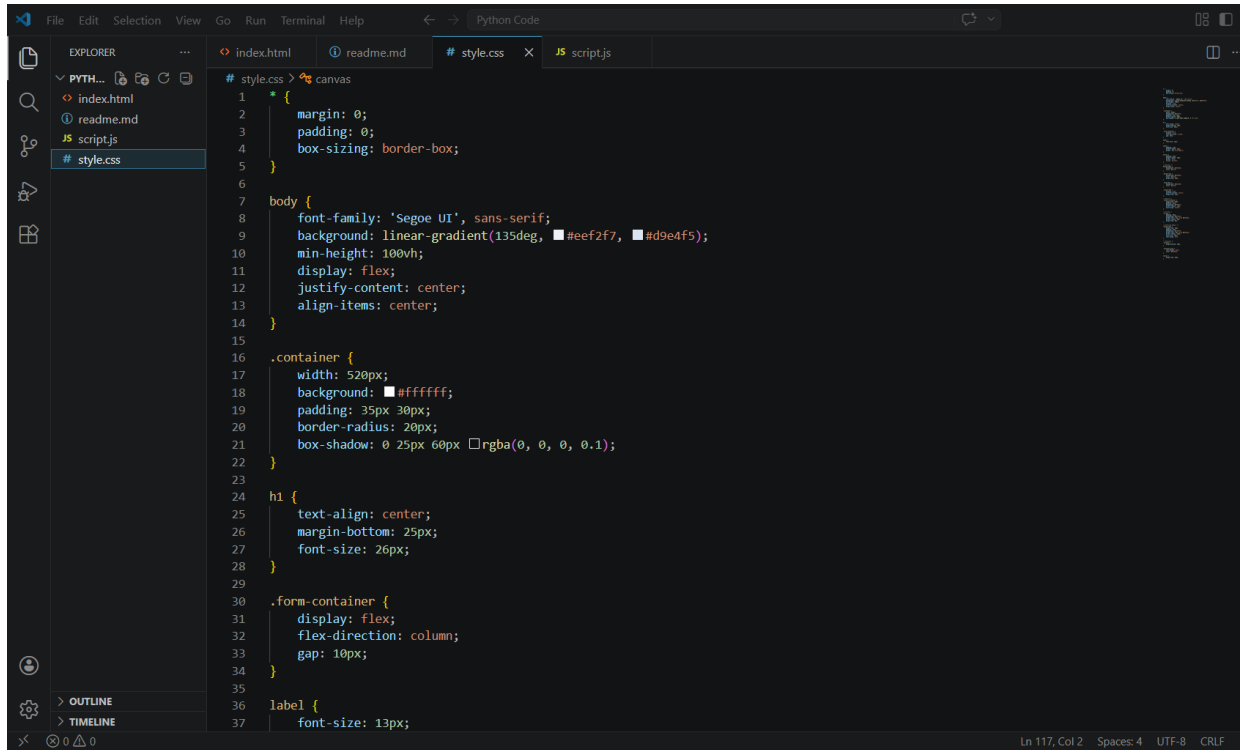
```
/* Sections */
```

```
.section {
    margin-bottom: 10px;
}
```

```
.section-title {
```

```
    font-weight: bold;
    color: #166534;
}

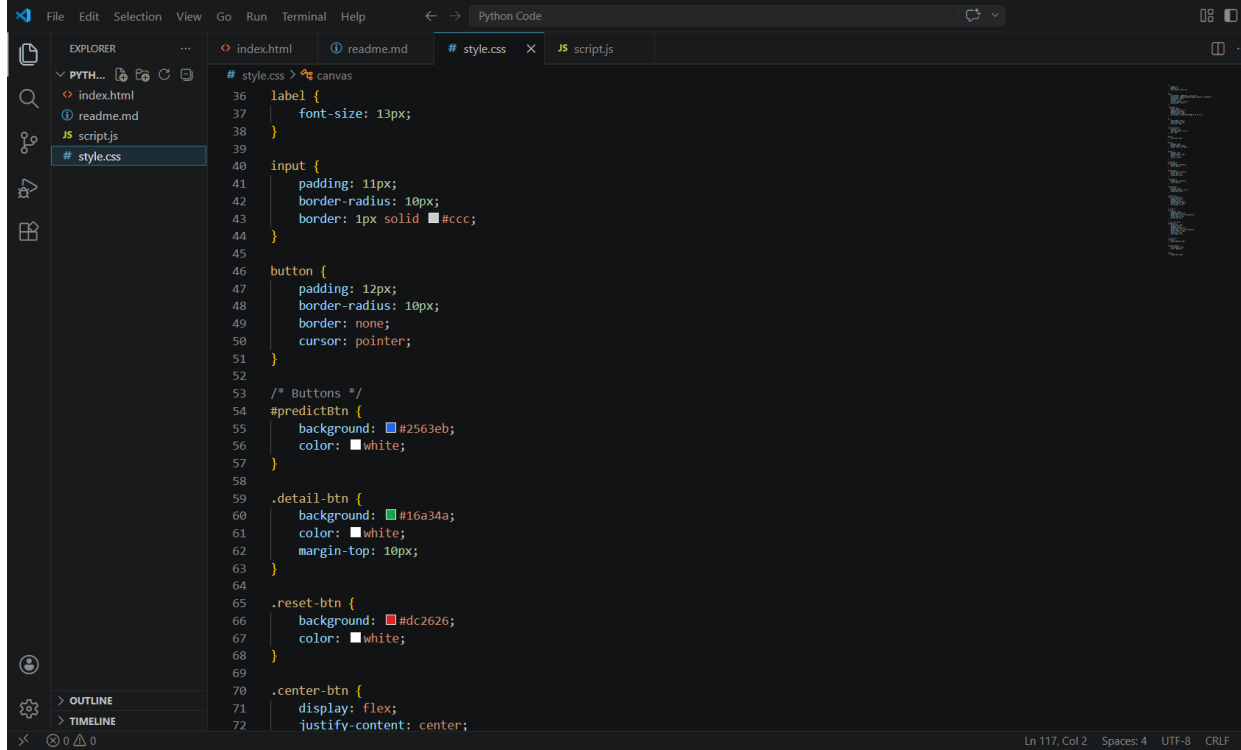
canvas {
    margin-top: 20px;
}
```



The image shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project named 'PYTHON' with files 'index.html', 'readme.md', 'script.js', and 'style.css'. The 'style.css' file is selected and its content is displayed in the code editor. The code defines styles for a canvas, body, container, h1, form-container, and label.

```
# style.css > canvas
1  * {
2    margin: 0;
3    padding: 0;
4    box-sizing: border-box;
5  }
6
7  body {
8    font-family: 'Segoe UI', sans-serif;
9    background: linear-gradient(135deg, #eef2f7, #d9e4f5);
10   min-height: 100vh;
11   display: flex;
12   justify-content: center;
13   align-items: center;
14 }
15
16 .container {
17   width: 520px;
18   background: #ffffff;
19   padding: 35px 30px;
20   border-radius: 20px;
21   box-shadow: 0 25px 60px rgba(0, 0, 0, 0.1);
22 }
23
24 h1 {
25   text-align: center;
26   margin-bottom: 25px;
27   font-size: 26px;
28 }
29
30 .form-container {
31   display: flex;
32   flex-direction: column;
33   gap: 10px;
34 }
35
36 label {
37   font-size: 13px;
```

Ln 117, Col 2 Spaces: 4 UTF-8 CRLF



The image shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project named 'PYTHON' with files 'index.html', 'readme.md', 'script.js', and 'style.css'. The 'style.css' file is selected and its content is displayed in the code editor. The code defines styles for a label, input, button, and various buttons.

```
# style.css > canvas
36 label {
37   font-size: 13px;
38 }
39
40 input {
41   padding: 11px;
42   border-radius: 10px;
43   border: 1px solid #ccc;
44 }
45
46 button {
47   padding: 12px;
48   border-radius: 10px;
49   border: none;
50   cursor: pointer;
51 }
52
53 /* Buttons */
54 #predictBtn {
55   background: #2563eb;
56   color: white;
57 }
58
59 .detail-btn {
60   background: #16a34a;
61   color: white;
62   margin-top: 10px;
63 }
64
65 .reset-btn {
66   background: #dc2626;
67   color: white;
68 }
69
70 .center-btn {
71   display: flex;
72   justify-content: center;
```

Ln 117, Col 2 Spaces: 4 UTF-8 CRLF


```
# style.css > canvas
70 .center-btn {
72   justify-content: center;
73   margin-top: 15px;
74 }
75
76 /* Result */
77 .result-container {
78   margin-top: 20px;
79   padding: 15px;
80   background: #f8f8fc;
81   border-radius: 12px;
82   text-align: center;
83 }
84
85 /* AI Box */
86 .ai-box {
87   padding: 10px;
88   background: #eff6ff;
89   border-left: 4px solid #2563eb;
90   border-radius: 8px;
91   margin-top: 8px;
92 }
93
94 /* Detailed Report */
95 .detail-box {
96   margin-top: 10px;
97   padding: 15px;
98   background: #fff0f4;
99   border-left: 5px solid #16a34a;
100  border-radius: 10px;
101  font-size: 13px;
102  text-align: left;
103 }
104
105 /* Sections */
106 .section {
107   margin-bottom: 10px;
```

```
# style.css > canvas
86 .ai-box {
90   border-radius: 8px;
91   margin-top: 8px;
92 }
93
94 /* Detailed Report */
95 .detail-box {
96   margin-top: 10px;
97   padding: 15px;
98   background: #fff0f4;
99   border-left: 5px solid #16a34a;
100  border-radius: 10px;
101  font-size: 13px;
102  text-align: left;
103 }
104
105 /* Sections */
106 .section {
107   margin-bottom: 10px;
108 }
109
110 .section-title {
111   font-weight: bold;
112   color: #166534;
113 }
114
115 canvas {
116   margin-top: 20px;
117 }
```

Result

To see the output, we have to click 'Go Live' which is located at the bottom-right corner of VS Code.

Go Live

After clicking, this page appears :-

Student Performance Dashboard

Study Hours

Attendance (%)

Assignments Completion (%)

Previous Marks (%)

Sleep Hours

Screen Time (hrs/day)

Predict Marks

Predicted Marks: --

Get Detailed Analysis

Marks

100

90

80

70

60

50

40

30

20

10

0

Reset Graph

Examples :-

1ST Attempt:-

Student Performance Dashboard

Study Hours

6

Attendance (%)

85

Assignments Completion (%)

80

Previous Marks (%)

85

Sleep Hours

7

Screen Time (hrs/day)

5

Predict Marks

Predicted Marks: 88.50

Excellent performance. Keep it up!

Get Detailed Analysis

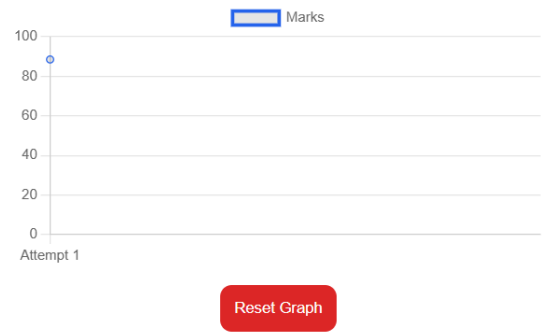
Performance Level: Excellent

Predicted Marks: 88.50

Strengths: - Good study hours - Good attendance - Healthy sleep - Controlled screen time

Weaknesses: - None

Recommendations: - Improve weak areas - Maintain consistency - Balance study and rest



2nd Attempt:-

Student Performance Dashboard

Study Hours

5

Attendance (%)

80

Assignments Completion (%)

85

Previous Marks (%)

80

Sleep Hours

7

Screen Time (hrs/day)

7

Predict Marks

Predicted Marks: 77.75

Trend: Declining

Excellent performance. Keep it up!

Get Detailed Analysis

Performance Level: Excellent

Predicted Marks: 77.75

Strengths: - Good study hours - Good attendance - Healthy sleep

⚠ Weaknesses: - High screen time

Recommendations: - Improve weak areas - Maintain consistency - Balance study and rest



Reset Graph

3rd Attempt:-

Student Performance Dashboard

Study Hours

8

Attendance (%)

85

Assignments Completion (%)

90

Previous Marks (%)

70

Sleep Hours

7

Screen Time (hrs/day)

6

Predict Marks

Predicted Marks: 94.25

Trend: Improving

Excellent performance. Keep it up!

Get Detailed Analysis

Performance Level: Excellent

Predicted Marks: 94.25

Strengths: - Good study hours - Good attendance - Healthy sleep

⚠ Weaknesses: - High screen time

Recommendations: - Improve weak areas - Maintain consistency - Balance study and rest

Marks

100

80

60

40

20

0

Attempt 1

Attempt 2

Attempt 3

Reset Graph

Testing Approach

To ensure the **Student Performance Predictor** works correctly, basic testing was done on all features of the system.

First, the input fields were checked by entering both valid and invalid values to make sure the system accepts correct data and shows an alert for missing or wrong inputs.

Then, the prediction feature was tested using different combinations of values to confirm that the calculated marks are accurate and always remain between 0 and 100.

The graph was tested by making multiple predictions to see if it updates properly and shows the correct performance trend (improving or declining).

The AI suggestions and detailed analysis were also checked to ensure they provide meaningful and relevant feedback based on the inputs.

Finally, the reset button was tested to make sure it clears all data and allows the user to start again easily.

Overall, the testing ensured that the system is accurate, user-friendly, and works smoothly without errors.

Future Enhancement

The **Student Performance Predictor** can be improved in many ways to make it more powerful and realistic. In the future, a real Machine Learning model can be added using a proper dataset. This will make the predictions more accurate instead of relying on a fixed formula.

Another improvement can be adding a login system so that each student can save their data and track their performance over time. This will make the system more personalized and useful.

The project can also include a feature to download the detailed analysis as a **PDF** report. This would help students keep a record of their performance and share it if needed.

More factors can be added in the future, such as stress level, subject difficulty, or exam preparation level, to make the prediction even more realistic.

In addition, the system can be expanded to compare multiple students or show class-level analysis, which would be helpful for teachers as well.

Overall, with these improvements, the project can be developed into a more advanced and practical system that can be used in real educational environments.

Conclusion

The Student Performance Predictor is a simple yet meaningful project that shows how technology can be used to understand and improve academic performance. By considering multiple factors such as study habits, attendance, sleep, and screen time, the system provides a more realistic view of how different aspects of a student's daily routine can impact their results.

Through this project, we were able to apply basic concepts of prediction and data analysis. Even though the system uses a simple formula instead of a fully trained machine learning model, it successfully demonstrates how predictions can be made using multiple inputs. This makes it a great starting point for understanding more advanced AI and Machine Learning techniques.

One of the most valuable parts of this project is that it goes beyond just predicting marks. It also provides helpful suggestions and a detailed analysis, allowing users to identify their strengths and weaknesses. The addition of graphs and trend analysis further improves the experience by making the results easy to understand and visually clear.

Overall, this project highlights how a well-designed system can combine prediction, visualization, and intelligent feedback into one platform. It is not only useful as a learning tool but also as a practical application that can guide students toward better performance. With further improvements, such as integrating real datasets and machine learning models, this project can be expanded into a more advanced and impactful system.

